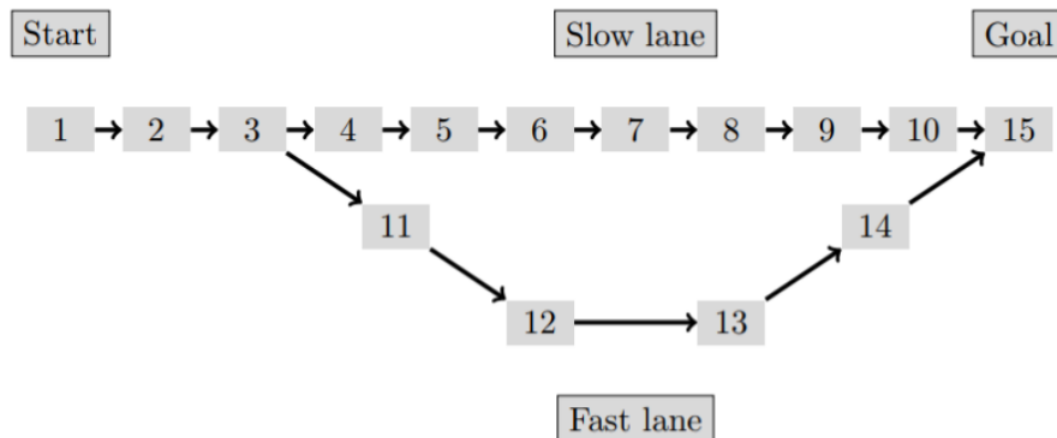

LINFO2275

First project

Group 2:

Gavin Hope –
Jakob Thibodeau –
Lore Lernoux 77562000 – MAP



Introduction

In this project, we present the implementation of a Markov Decision Process to solve a Snakes and Ladders game. To do so, we implemented the value iteration algorithm to find the optimal policy for the game. This policy specifies, for each tile, the best dice to use in order to minimize the expected number of turns to reach the goal (tile 15). Afterwards, we compared this optimal policy to other suboptimal policies, such as fixed-dice policies, to see how they perform in terms of expected number of turns to win the game. Both theoretical results from value iteration and empirical results from Monte Carlo simulations are analyzed, presented and compared across different trap layouts, for both game configurations.

Theoretical background

The game is modeled as a Markov Decision Process. As seen during the lectures, it is defined by

- a set of states S , which in our case corresponds to the tiles of the board, i.e. $S = \{1, 2, \dots, 15\}$;
- a set of actions A , which corresponds to the choice of dice to roll, i.e. $A = \{1, 2, 3, 4\}$;
- a transition function $P(s' | s, a)$, which gives the probability of reaching state s' from state s after choosing action a . These probabilities are determined by the stochastic outcomes of the dice, the branching structure of the board, and the activation of traps;
- a cost function $c(s | a)$, which gives the cost when taking action a in state s . In our case, we can define the cost as 1 for each turn taken, and 0 for reaching the goal (tile 15).

The goal is to find an optimal policy π^* which specifies the best dice to use at each tile in order to minimize the expected number of turns to reach tile 15 from any initial state. Let $V(s)$ be this total expected number of turns which

respects Bellman's equation with as condition $V(15) = 0$:

$$V(s) = \min_{a \in A} \left[c(s | a) + \sum_{s' \in S} P(s' | s, a) V(s') \right] \quad (1)$$

This optimal expected number of turns can be computed using the value iteration algorithm, which iteratively updates $V(s)$ for each state until convergence. The optimal policy is then obtained by selecting, for each state, the action that minimizes the Bellman equation.

To validate the theoretical results, we performed Monte Carlo simulations. To do so, we simulated 1000 games from the initial state until the goal state following a policy, and we computed the empirical average number of turns. We also computed the standard deviation of these estimates. These are then compared to the theoretical values obtained through value iteration in order to validate the correctness of the optimal policy.

Implementation description

Trap layouts

We chose to use 6 different trap layouts to compare the optimal policy in different settings. These layouts are the following :

- **No traps**: this layout has no traps, which means that the player can only move forward and never go back. This is the simplest layout and serves as a baseline for comparison.
- **Few traps**: this layout has a few traps scattered throughout the board.
- **Many traps**: this layout has many traps scattered throughout the board, which makes it more difficult to win the game.
- **Evil fast lane**: this layout has traps only on the fast lane.
- **Two-in-a-row traps**: this layout has two-times two traps in a row, which makes it more difficult to avoid them.

- **Back-to-three traps:** this layout has traps that send the player back to tile 3, which is the branching tile.

Sub-optimal strategies

We chose to implement five sub-optimal strategies to compare with the optimal policy. These strategies are the following :

- **One dice only:** for each of the 4 dices, we made a policy that always chooses that dice. This gives us 4 sub-optimal policies to compare with the optimal one.
- **Uniform random strategy:** this policy chooses a dice uniformly at random, independently of the current tile. This is a baseline strategy.

Results

Theoretical vs Empirical Results: Optimal Strategy

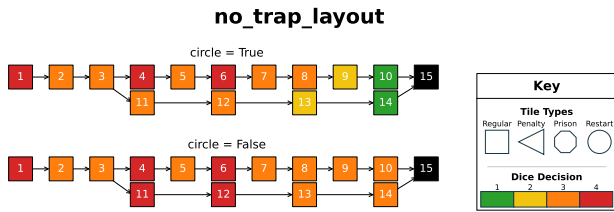


Figure 1: Expected number of turns to win the game for the no traps layout.

Figure 1 shows the optimal policy in the case where no traps are present, in both the circular and linear game structures. The strategies are slightly different in the two structures, as we can see that for tiles closed to the goal, the circle structure favors safer dices as we must stop exactly on the goal, while the linear structure favors faster dices as we can overshoot the goal. For the rest of the tiles, it favors riskier dices as there are no traps to worry about, and the goal is to reach tile 15 as fast as possible.

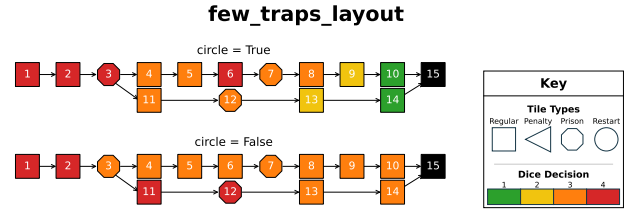


Figure 2: Expected number of turns to win the game for the few traps layout.

Figure 2 is more interesting as the game is more complex with the presence of traps. We can see that the optimal policy is again different in the two structures.

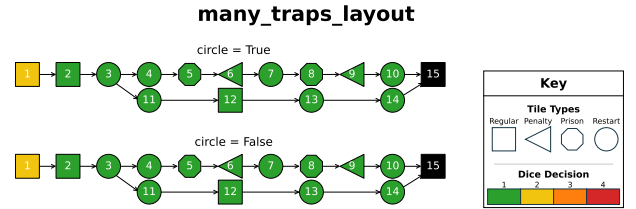


Figure 3: Expected number of turns to win the game for the many traps layout.

Figure 3 shows the optimal policy for the many traps layout, which is even more complex than the few traps layout. Here, the optimal policies are the same: they both favor the safest dice (dice 1) for most of the tiles, which is not surprising given the high number of traps..

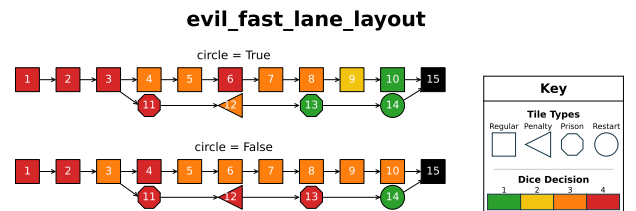


Figure 4: Expected number of turns to win the game for the evil fast lane layout.

Figure 4

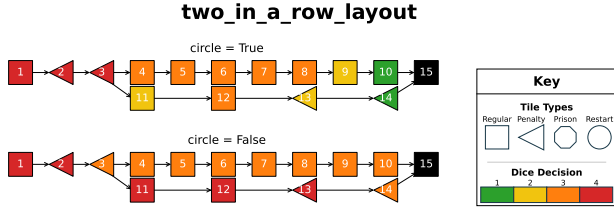


Figure 5: Expected number of turns to win the game for the two-in-a-row traps layout.

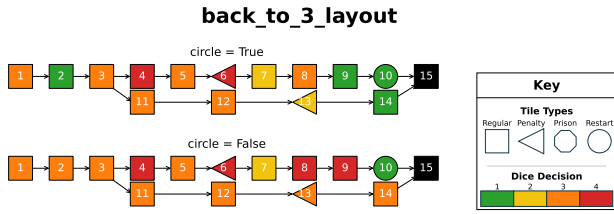


Figure 6: Expected number of turns to win the game for the back-to-three traps layout.

Theoretical vs Empirical Results: Sub-optimal Strategies

We compared the optimal policy with the 5 sub-optimal strategies by simulating 1000 games for each policy and game structure (circle or not) and computing the average number of turns to win the game, i.e. the expected number of turns needed to reach tile 15 from tile 1. For each trap layout, we represented the game and the optimal policy by coloring the tiles according to the dice that should be used on that tile.

Conclusion